DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 A

ALGORITHMIC DESIGN THROUGH PSEUDOCODE

FRACTIONAL KNAPSACK

Algorithm: Fractional Knapsack

```
FractionalKnapsack(Items, Capacity)
    Sort Items in descending order of value/weight ratio
    TotalValue = 0
    For each Item in Items
        If Capacity == 0
            Break
        End If
        If Item.Weight <= Capacity
            TotalValue = TotalValue + Item.Value
            Capacity = Capacity - Item.Weight
        Else
            Fraction = Capacity / Item.Weight
            TotalValue = TotalValue + (Fraction * Item.Value)
            Capacity = 0
        End If
    End For
    Return TotalValue
End Algorithm
```

JOB SEQUENCING WITH DEADLINES

Algorithm: Job Sequencing with Deadlines

```
JobSequencing(Jobs)
    Sort Jobs in descending order of Profit
    MaximumDeadline = Maximum deadline among Jobs
    Create Slots[1...MaximumDeadline]
    Initialize all Slots as EMPTY
    TotalProfit = 0
    For each Job in Jobs
        For Time = Job.Deadline down to 1
            If Slots[Time] is EMPTY
                Slots[Time] = Job
                TotalProfit = TotalProfit + Job.Profit
                Break
            End If
        End For
    End For
    Return Slots, TotalProfit
End Algorithm
```

ZERO-ONE KNAPSACK USING DYNAMIC PROGRAMMING

Algorithm: 0/1 Knapsack

```
Knapsack(Weight, Value, n, Capacity)
    Create DP[0...n][0...Capacity]
    For i = 0 to n
        DP[i][0] = 0
    End For
    For w = 0 to Capacity
        DP[0][w] = 0
    End For
    For i = 1 to n
        For w = 1 to Capacity
            If Weight[i] <= w
                DP[i][w] = Maximum(
                    Value[i] + DP[i-1][w-Weight[i]],
                    DP[i-1][w]
                )
            Else
                DP[i][w] = DP[i-1][w]
            End If
        End For
    End For
    Return DP[n][Capacity]
End Algorithm
```

WORD BREAK PROBLEM – DIAGNOSE AND REPAIR POOR PSEUDOCODE

Corrected Algorithm: Word Break

```
WordBreak(Text, Dictionary)
    n = Length(Text)
    Create DP[0...n]
    Initialize all DP values as FALSE
    DP[0] = TRUE
    For i = 1 to n
        For j = 0 to i-1
            If DP[j] == TRUE AND
               Text[j...i-1] exists in Dictionary
                DP[i] = TRUE
                Break
            End If
        End For
    End For
    Return DP[n]
End Algorithm
```

SHORTEST PATH ALGORITHMS

Algorithm: Dijkstra's Algorithm

```
Dijkstra(Graph, Source)
    For each Vertex in Graph
        Distance[Vertex] = INFINITY
        Visited[Vertex] = FALSE
    End For
    Distance[Source] = 0
    While there are unvisited vertices
        u = Unvisited vertex with minimum Distance
        If Distance[u] == INFINITY
            Break
        End If
        Visited[u] = TRUE
        For each Neighbor v of u
            If Visited[v] == FALSE AND
               Distance[u] + Weight(u,v) < Distance[v]
                Distance[v] = Distance[u] + Weight(u,v)
            End If
        End For
    End While
    Return Distance
End Algorithm
```

Algorithm: Bellman-Ford Algorithm

```
BellmanFord(Graph, Source)
    For each Vertex in Graph
        Distance[Vertex] = INFINITY
    End For
    Distance[Source] = 0
    Repeat |V| - 1 times
        For each Edge (u, v) with Weight w
            If Distance[u] != INFINITY AND
               Distance[u] + w < Distance[v]
                Distance[v] = Distance[u] + w
            End If
        End For
    End Repeat
    For each Edge (u, v) with Weight w
        If Distance[u] != INFINITY AND
           Distance[u] + w < Distance[v]
            Return "Negative Cycle Exists"
        End If
    End For
    Return Distance
End Algorithm
```

Algorithm: Floyd-Warshall Algorithm

```
FloydWarshall(Graph)
    Initialize Distance[i][j] using edge weights
    For each Vertex i
        Distance[i][i] = 0
    End For
    For k = 1 to |V|
        For i = 1 to |V|
            For j = 1 to |V|
                Distance[i][j] = Minimum(
                    Distance[i][j],
                    Distance[i][k] + Distance[k][j]
                )
            End For
        End For
    End For
    Return Distance
End Algorithm
```

ENERGY-AWARE WEIGHTED INTERVAL SCHEDULING

Algorithm: Weighted Interval Scheduling

```
WeightedIntervalScheduling(Jobs)
   Sort Jobs in ascending order of Finish Time
   For i = 1 to n
      p[i] = Index of the latest job j < i
            such that Finish[j] <= Start[i]
      If no such job exists
         p[i] = 0
      End If
   End For
   DP[0] = 0
   For i = 1 to n
      Include = Energy[i] + DP[p[i]]
      Exclude = DP[i-1]
      DP[i] = Maximum(Include, Exclude)
   End For
   SelectedJobs = EMPTY
   i = n
   While i > 0
      If DP[i] == Energy[i] + DP[p[i]]
         Add Job[i] to SelectedJobs
         i = p[i]
      Else
```

```
            i = i - 1
        End If
    End While
    Reverse SelectedJobs
    Return DP[n], SelectedJobs
End Algorithm
```